LoRA Meets Tables: Boosting Table Understanding with Parameter-Efficient Fine-Tuning

Hanze Meng
hmeng99@cs.ubc.ca
University of British Columbia
Vancouver, British Columbia, Canada

Jin Wang
jinwang18@asu.edu
Arizona State University
Tempe, Arizona, USA

Rachel Pottinger
rap@cs.ubc.ca
University of British Columbia
Vancouver, British Columbia, Canada

ABSTRACT

Table understanding is a fundamental problem in data management, as it is the basis of core tasks such as data integration, discovery, and cleaning. There are many different table understanding tasks, such as Column Type Annotation (CTA) and Column Property Annotation (CPA); improving these tasks enables users of relational and web tables to better understand and explore their data.

There have been many different techniques for solving table understanding tasks such as CTA and CPA. Most recent works focus on prompt engineering over Large Language Models (LLMs). However, the performance of prompt-based approaches is always suboptimal, since they are not able to fully capture and use table-aware information, which is very different in characteristic from prior table understanding approaches. While fine-tuning LLMs could be an intuitive solution to address such issues, its huge overhead and cost makes this option less feasible in practice.

In this paper, we propose TSLoRA, a novel method of solving table understanding tasks which is the first parameter-efficient fine-tuning approach to support a variety of table understanding tasks rather than focusing on just one. This improves the needs of data management professionals by enabling them to use a much cheaper general purpose method rather than having to use several different methods, each of which must be fine-tuned at great expense. We further develop several techniques to incorporate structural information from the table and encode the alignment between essential substructures of the table and task instructions; this helps to bridge the loses of moving from more traditional data management techniques to using more recent LLM-based techniques. Experimental results on two large table corpora demonstrate that our proposed method consistently achieves better effectiveness on multiple table understanding tasks than state-of-the-art methods, both those using LLMs and those using other methods, and yet introduces trivial fine-tuning overhead.

PVLDB Reference Format:
Hanze Meng, Jin Wang, and Rachel Pottinger. LoRA Meets Tables: Boosting Table Understanding with Parameter-Efficient Fine-Tuning. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

The relational web table is the fundamental format to organize structured datasets on the world wide web. Over the past decades, many large-scale table corpora (e.g., [4, 29, 37]) were constructed, which have been recognized as valuable resources for many applications, both for end users and experimental analysis. Unfortunately, such corpora can be difficult for users to understand. *Table understanding* is the umbrella term used to describe efforts that annotate and process data in tables. It plays a significant role in analyzing such table corpora so as to use the rich set of knowledge stored in them to support a multitude of downstream tasks [2, 18, 40], such as data integration, data discovery and question answering. A typical category of table understanding tasks is table interpretation [12, 45, 61], which mainly infers the meta information about tables, such as column headers and relationships between columns.

There is a long stream of research efforts about table understanding in the data management community. Earlier studies (e.g., [30, 68, 70]) relied on hand-crafted features and employed classic machine learning or rule-based methods. A more recent paradigm is fine-tuning Transformer-based Pre-trained Language Models (PLMs) such as BERT [13]. While these approaches [12, 17, 45, 53] achieve very promising results, each task requires a separate training set with a non-trivial amount of labeled instances to fine-tune the model. As a result, there is a substantial waste in the efforts of data and model engineering. Recently, Large Language Models (LLMs) have been widely adopted in various table-related tasks (e.g., [3, 39, 72]). Recent pre-trained LLMs such as GPT [49], LLaMA [44] and Gemini [56] have shown superior ability in understanding human instructions as well as generating structured output. To use LLMs for table understanding, previous studies [20, 25, 33] developed effective *prompt engineering* strategies by providing the (partial) table together with instructions that describe the task and then collected the output of LLMs as the results. Such methods could avoid the extra cost in data and model engineering of PLM-based methods: they could support different table understanding tasks by just slightly modifying the prompt templates as the input to the same LLM. However, in many cases their performance is suboptimal on many specific tasks because their capability in learning the table-aware information is somewhat limited.

To overcome the limitations of previous PLM and LLM-based methods, a straightforward methodology would be fine-tuning the LLMs over table corpora and then performing prompt engineering over it. However, the overhead could be tremendous comparing with that of fine-tuning the PLMs. For example, as illustrated in Figure 1, full supervised fine-tuning (SFT) of a 3B and a 4B model requires on average 35.9 GB and 37.8 GB of GPU memory with small batch sizes of only 4 and 2, respectively. Such memory demands

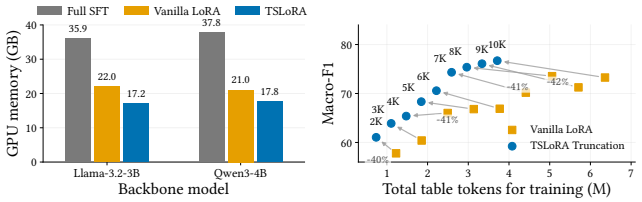


Figure 1: Comparison of GPU memory consumption and training-token efficiency among different fine-tuning strategies on Llama-3.2-3B [44] and Qwen3-4B [52]. While vanilla LoRA substantially reduces memory usage compared with full-model SFT, TSLoRA further decreases the memory by approximately 19% across the two models. Comparing vanilla LoRA with TSLoRA across different numbers of training instances (2K to 10K), TSLoRA consistently achieves higher Macro-F1 while processing approximately 40%–42% fewer table tokens than vanilla LoRA.

make task-specific adaptation costly, limiting the practicality of fine-tuning LLMs for table understanding. Meanwhile, leveraging the techniques of Parameter-Efficient Fine-Tuning (PEFT) [16] can ameliorate this issue. Among them Low-rank Adaptation (LoRA) [27] is a popular technique that can decrease the costs of fine-tuning the LLMs to support table understanding tasks.

LoRA's key idea is only updating a small fraction of trainable parameters while keeping the majority of parameters in the LLM unchanged during fine-tuning. This way, the cost of fine-tuning LLMs can be significantly reduced. For example, as shown in Figure 1, vanilla LoRA reduces the average GPU memory usage to 22.0 GB and 21.0 GB for the 3B and 4B models respectively, achieving a reduction of approximately 42% compared with full SFT.

Such substantial savings demonstrate the potential of LoRA as a practical approach to adapting LLMs for data management tasks. However, to reach the goal of adopting LoRA for table understanding tasks such as Column Type Annotation (CTA) and Column Property Annotation (CPA), there are still two research challenges to be overcome: (i) to obtain a universal solver for table understanding, we need to properly prepare the training dataset for various tasks from large-scale table corpora; and (ii) to improve the performance of table understanding tasks, we need to incorporate sufficient context and structural information from the table in the fine-tuning process, while the vanilla LoRA method is designed for general fine-tuning process and does not take care of that.

This paper proposes **T**able-token efficient and **S**tructure aware **LoRA** (TSLoRA), an end-to-end framework for table understanding based on LoRA. In order to prepare the training data for the LoRA based fine-tuning process over large-scale table corpora to support various table understanding tasks, we devised an instruction tuning process. To be more specific, we defined the training instances as a prompt with the input (partial) table, task instructions and the question; the ground truth is defined as the answer of the given question. In this way, we can incorporate multiple table understanding tasks within the same fine-tuning process. Since the table can be very large, we further developed some truncation techniques to save the number of table tokens involved in the training instances. In addition, we proposed a two-stage training strategy: the first stage enables the model to gain a better understanding of the positional information within the table; On the basis of that, the second stage further encodes the alignment between the instructions of relevant substructures in the table with low-rank embeddings of LoRA and upscales them to integrate with LLM token embeddings.

We conducted an extensive set of experiments on two large-scale table corpora, WikiTables and WDC, over several table understanding tasks on two open sourced LLMs: Llama and Qwen. The experimental results demonstrated that TSLoRA can outperform state-of-the-art methods on CTA and CPA tasks by an obvious margin. Additionally, TSLoRA can also support more general table understanding tasks, such as those in the recently released Multi-Task Table Understanding and Reasoning Benchmark (MMTU) [62]. The entire fine-tuning process can be carried out on a partition of an NVIDIA H100 GPU, without requiring multi-GPU training.

To sum up, we made the following contributions in this paper:

- We propose the TSLoRA framework to support table understanding tasks by fine-tuning LLMs with LoRA. To the best of our knowledge, this is the first work to develop parameter-efficient fine-tuning techniques for general table-related tasks.
- To construct the table understanding training datasets from large-scale corpora for the fine-tuning process, we develop an instruction-tuning approach. We also develop effective truncation mechanisms to reduce the number of tokens in the training instances.
- To incorporate the essential contextual and structural information necessary for table understanding, we devise a novel two-stage training strategy to improve the LoRA fine-tuning process.
- We conduct experiments on two large-scale table corpora to illustrate the superiority of our proposed techniques in the aspects of both improving the effectiveness and saving the cost for table understanding tasks.

The rest of this paper is organized as follows: Section 2 provides background knowledge; Section 3 introduces the overall framework of our proposed solution; Section 4 proposes techniques to prepare the training data; Section 5 develops the two-stage strategy for the fine-tuning process; Section 6 reports the experimental results; Section 7 surveys related work. Section 8 concludes.

2 BACKGROUND

2.1 Table Understanding

Table understanding, i.e., an umbrella of data management tasks that analyze or increase data and metadata about a given table, is essential in providing important annotations to different granularity of tables, such as cell, row, column, column pairs, and the whole table. It can also potentially encompass a broad spectrum of tasks such as table transformation, augmentation and matching. This paper uses two tasks, Column Type Annotation (also known as semantic type detection) [12, 20, 25, 45, 53, 61, 68] and Column Property Annotation (also known as relationship extraction) [12, 45, 53, 61] as specific, motivating problems within the table understanding umbrella. As they are the most popular problems in the previous studies, we believe it would be persuasive to highlight the superiority of our proposed techniques by comparing our methods with the state-of-the-art methods on these two tasks. The formal definitions

of the two tasks that we concentrate on in this paper are shown in Definition 2.1 and 2.2, respectively.

Definition 2.1 (Column Type Annotation (CTA)). Given a table, T, and a set of semantic types, L, CTA identifies a type label, $l \in L$, for each column, $C \in T$, so that each cell in C has the same semantic type.

Definition 2.2 (Column Property Annotation (CPA)). Given a table, T, a set of relation types, R, and a pair of columns, $C_i, C_j \in T$ ($i \neq j$), CPA identifies a relation label, $r \in R$, so that r describes the relationship between all pairs of cells in the two columns.

While this paper concentrates on CTA and CPA to concretely show our overall techniques, our techniques can also be generalized to other table understanding tasks, all of which are used in downstream data management applications. We further illustrate the performance on other table understanding tasks in Section 6.

2.2 Low-Rank Adaptation

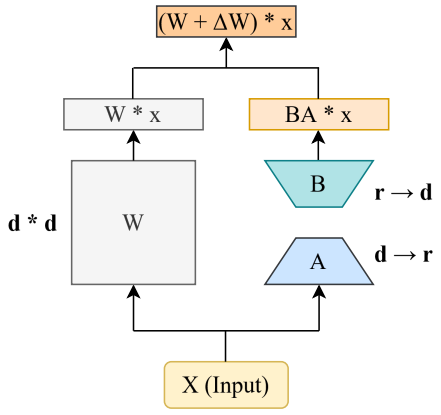


Figure 2: Illustration of Low-Rank Adaptation (LoRA)

Research on many data management tasks, including table understanding tasks, currently uses LLMs. These LLMs have many parameters. Parameter-efficient fine-tuning (PEFT) [16] is an alternative to full-model fine-tuning that reduces the cost of adapting large pre-trained models to other downstream tasks. Instead of updating all of the model's parameters, which can be computationally expensive, PEFT methods keep the pre-trained model weights frozen and introduce a small number of task-specific trainable parameters. In this way, the training time and memory consumption can be significantly reduced, while still enabling effective task adaptation. Thus, the primary goal is to decrease the costs (in time, tokens, and other metrics) of tasks that use LLMs while still maintaining the performance of state-of-the-art methods.

In this work, we employ *Low-Rank Adaptation (LoRA)* [27], a popular technique in PEFT, as the cornerstone of our framework. The high level idea of LoRA is shown in Figure 2: given a matrix of pretrained weights $W \in \mathbb{R}^{d \times d}$, LoRA keeps W frozen and introduces a low-rank adaptation for efficient weight updates through two smaller matrices: $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ where $r \ll d$, reducing the number of trainable parameters on W from d^2 to $2rd$. For example, with $d = 4096$ and a LoRA rank of $r = 16$, the number of trainable parameters decreases from approximately 16.8 million to 0.13 million. The update is then computed as $\Delta W = BA$. The adapted weights are obtained as $W + \frac{\alpha}{r}\Delta W$, where $\frac{\alpha}{r}$ is a scaling factor. Instead of learning a full $d \times d$ weight update matrix, LoRA learns a pair of much smaller matrices whose product approximates the desired update. As a result, it allows models to be efficiently adapted to new tasks with minimal extra computation or storage.

3 OVERALL TSLORA FRAMEWORK

We show the overall architecture of our proposed TSLoRA table understanding framework in Figure 3. The *Data Loader* module digests the input raw table corpora and conducts essential cleaning and transformation. The *Training Data Preparation* module constructs training instances for different table understanding tasks from the tables processed by the *Data Loader*. We propose a training data preparation approach by making these table understanding instances follow a unified instruction-tuning format, allowing different tasks to be incorporated into a single fine-tuning process. Additionally, since many tables are very large, containing thousands of rows, using the entire table can be expensive. Thus, we also develop effective truncation techniques to reduce the number of rows of tables that are fed to the LLMs, which reduces the cost (i.e., tokens) required, without greatly impacting effectiveness. More details about the Training Data Preparation module are described in Section 4. The *Tuning Processor* module optimizes the LoRA fine-tuning process by incorporating contextual and structural information about the tables with a two-stage training strategy, which is described in Section 5. Finally, the *Evaluator* module evaluates the obtained model on various downstream tasks to report the performance. Together, all of these data management-specific optimizations allow us to show that TSLoRA performs better than previous table understanding work, as we show in Section 6.

4 TABLE UNDERSTANDING TRAINING DATA PREPARATION

4.1 Table Understanding Training Data Definition

The first step in fine-tuning the LLM for table understanding tasks over large-scale table corpora is to obtain the training datasets. To ensure the versatility of the output model for table understanding tasks, we need to enable different tasks to be trained within the same fine-tuning process. We developed an instruction-tuning method to satisfy this requirement, thus achieving a universal solver for table understanding tasks. The core idea is that instead of training a table encoder as previous PLM based table understanding methods did, we trained a decoder-only LLM to generate predictions from prompt input that includes serialized tables.

In our table understanding setting, a *training instance* is a prompt containing the information required for the LLM to make a prediction, while the *ground truth* is the corresponding expected answer. Each training instance consists of a prompt with three components:

- Instruction: describe the task requirement to the model.
- Table: provide the serialized table including both the targeted substructure of the table, as well as the contexts.

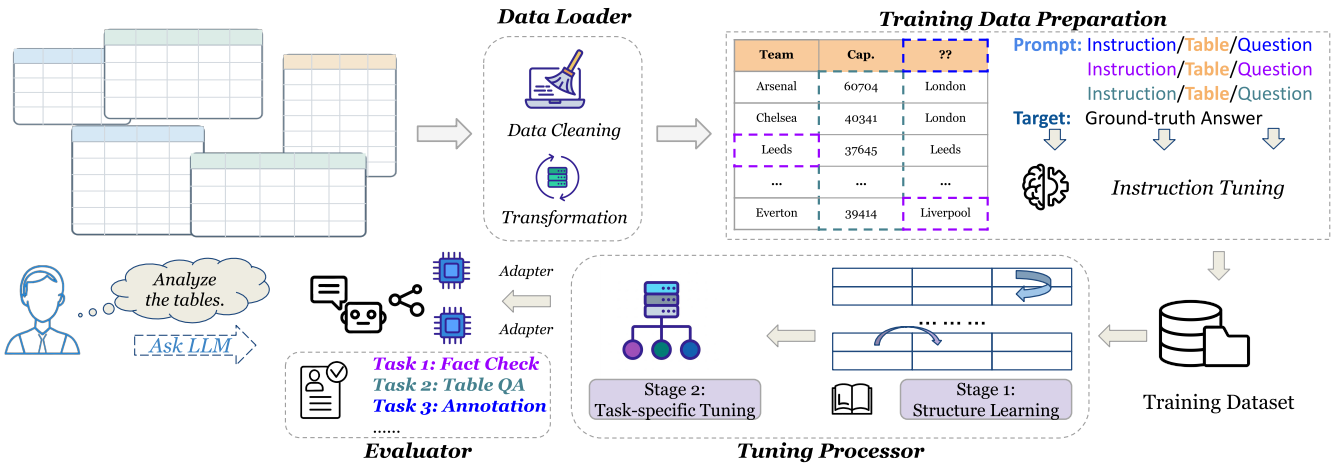


Figure 3: The Overall System Architecture

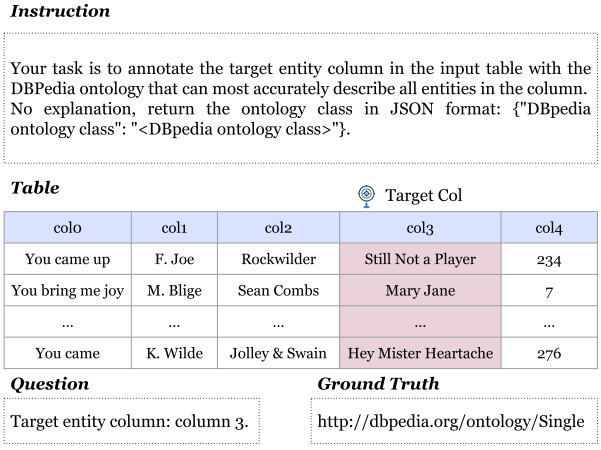


Figure 4: Illustration of an instruction-tuning instance for column type annotation. It consists of a task instruction, a serialized table, and a task-specific question identifying the target column. The corresponding DBpedia ontology class serves as the ground-truth output for supervised fine-tuning.

- Question: the table understanding question for the model to answer

Figure 4 provides an example of a training instance for CTA and its corresponding ground-truth.

To construct the prompt, we serialize each input table into a Markdown-style textual representation, where columns are separated by vertical bars and rows are separated by line breaks. The serialization begins with a header containing the column identifiers, followed by the table contents in their original row and column order. With such a definition of a training instance, it is relatively easy to extend the proposed framework to more table understanding tasks: when introducing a new table understanding task, we can simply change the instruction and question part accordingly to create the training instances for it. It is worth noting that these techniques are specific to the data management task of table understanding; they are tools that are being used to solve those tasks—they are not general purpose machine learning techniques.

4.2 Table Truncation Setup

So far we have described how to create a training instance for a given table corpus and downstream table understanding tasks. The next step is to decide how to prepare the training datasets for fine-tuning. This is critical, since using a naïve approach will result in a potentially very high overhead. Although the standard LoRA-based fine-tuning will ensure that the number of parameters trained is significantly lower than training all of the parameters, the hardware cost (e.g., GPU hour and memory usage) could still be substantial since the input tables could be very large. To address this issue, we developed novel, effective techniques to truncate the contents of the tables to be included in the training instances, while maintaining the effectiveness of the overall training process. Thus, our work allows for similar results to those of much more expensive table understanding methods, but at a much lower cost.

The prerequisite for truncation is to ensure the integrity of the prompts by protecting essential information for the model to answer the given table understanding questions. Rather than truncating tables solely based on their size or position, we preserve the substructures essential to the downstream table understanding task while selecting additional content to retain the broader table context under a limited token budget. Since different table understanding tasks may require preserving different parts of a table, we allow users to specify task-dependent truncation constraints to ensure that essential information is retained for their tasks. Specifically, a task-level policy determines whether truncation over specific units, e.g. row and column, is permitted; while instance-level metadata identifies the target substructures that must be preserved. For example, in CPA, which predicts the relationship between two columns, the two target columns must be retained as non-empty ones to preserve the relationship being annotated.

We implement these user-specified constraints through a metadata-guided applicability and protection mechanism as follows: Let T be a table with n data rows and m columns, with row and column index sets $V_r = \{1, \ldots, n\}$ and $V_c = \{1, \ldots, m\}$. The task can supply explicit metadata identifying target row indices, column indices, or cell coordinates, which we define as protected sets $P_r \subseteq V_r$ and $P_c \subseteq V_c$. Let $\mathcal{A}_r \subseteq V_r$ and $\mathcal{A}_c \subseteq V_c$ denote the indices of the retained rows and columns, and let $T[\mathcal{A}_r, \mathcal{A}_c]$ denote the sub-table they induce. The selection must satisfy $P_r \subseteq \mathcal{A}_r$ and $P_c \subseteq \mathcal{A}_c$.

Algorithm 1 summarizes how the task-specific constraints determine whether a table should be truncated and, if so, whether rows, columns, or both can be removed. When truncation is unnecessary or not permitted, the original table is retained. Let $b_r, b_c \in \{YES, NO\}$ indicate whether row and column truncation are permitted, respectively. These flags are determined by the user-specified task policy, G, independently of the protected regions identified by instance-level metadata, M. And the selected rows and columns will retain their original relative order in the training instance.

Algorithm 1: Metadata-guided table truncation

Input: T, metadata M, table retention policy G, budget β
if M does not specify a target region **then**
  return T
$(P_r, P_c) \leftarrow \text{Targets}(M)$
$(b_r, b_c) \leftarrow \text{PermittedTrunc}(G)$
if T fits within β or $\neg b_r \wedge \neg b_c$ **then**
  return T
$\mathcal{A}_r, \mathcal{A}_c \leftarrow V_r, V_c$
if b_r **then**
  $\mathcal{A}_r \leftarrow \text{RowSelect}(T, P_r, \beta)$
if b_c **then**
  $\mathcal{A}_c \leftarrow \text{ColumnSelect}(T, P_c, \beta)$
return $T[\text{sort}(\mathcal{A}_r), \text{sort}(\mathcal{A}_c)]$

4.3 TSLoRA Detailed Approach to Table Understanding

With the above design objectives and protection constraints, we now propose the detailed solution for table truncation. Since the goal is to ensure both *training-token efficiency* and *downstream table understanding task performance*, we need to reduce the number of serialized table tokens processed during training while keeping a high predictive quality of the model on the target task. However, directly evaluating downstream performance for every candidate truncation is computationally impractical. Thus, we instead develop a table selection strategy that preserves task-relevant information and representative table content. We primarily focus the truncation problem formulation on row dimension, as tables with large numbers of rows are a common source of excessive training tokens. Column-wise selection can be handled separately following a similar idea, as discussed later in this section.

Suppose the table T is regarded as a sequence of rows, $T = (r_1, \ldots, r_n)$, with index set $V = \{1, \ldots, n\}$. Then the *table truncation problem* can be formally defined as Definition 4.1.

Definition 4.1 (Table Truncation). Given a table T and the row budget K, the problem is to select a set of rows $\mathcal{A} \subseteq V$ to include in the training instance, where $|\mathcal{A}| = \min(K, n)$. The goal is to determine a row-priority ordering $\pi_T = (\pi_1, \ldots, \pi_n)$ from which the retained set $\mathcal{A}$ is selected.

When $\mathcal{A}$ is used in the training instance, the retained rows are restored to their original table order before serialization to conserve the semantics. The row budget K can be determined empirically to achieve the desired reduction in serialized table tokens across the training dataset.

4.3.1 Optimization Objectives. To decide such a row-priority ordering, we measure the similarity between table rows using Term Frequency-Inverse Document Frequency (TF-IDF) denoted by C, treating each row i as a textual document, d_i, whose content depends on the coverage variant described below. We represent d_i as an ℓ_2-normalized TF-IDF vector of character grams, z_i. The similarity between documents d_i and d_j is denoted by s_{ij}, and the matrix S collects all pairwise similarities:

$$z_i = \frac{C(d_i)}{\|C(d_i)\|_2}, \qquad s_{ij} = z_i^\top z_j, \qquad S = [s_{ij}]_{i,j=1}^{n}. \tag{1}$$

For nonzero vectors, s_{ij} is cosine similarity. The `coverage` variant constructs d_i from the predetermined focus field or fields, whereas `coverage_row` constructs d_i from all cells in row i.

We use these similarity scores to select rows that collectively represent the original table. For a retained set $\mathcal{A} \subseteq V$, we measure its coverage by summing the similarity between each original row and its most similar retained row. We then select the set $\mathcal{A}^\star$ that maximizes this coverage score:

$$\begin{aligned} F_{\text{cov}}(\mathcal{A}) &= \sum_{i \in V} \max_{j \in \mathcal{A}} s_{ij}, \\ \mathcal{A}^\star &\in \operatorname*{argmax}_{\mathcal{A} \subseteq V, |\mathcal{A}| = K} F_{\text{cov}}(\mathcal{A}). \end{aligned} \tag{2}$$

The term $\max_{j \in \mathcal{A}} s_{ij}$ measures how well original row i is represented by its most similar retained row. $\bar{K}$ is the minimum of row budget K and the number of rows n. This objective favors selecting complementary rows that collectively represent the table, since retaining nearly identical rows provides little additional coverage. Intuitively, the objective is to select $\bar{K}$ representative rows such that every original row can be represented by a sufficiently similar selected row. It can be formulated as an assignment problem: we decide which rows to retain, and assign each original row to one of the retained representatives, maximizing the overall similarity:

$$\begin{aligned} \max_{x,y} \quad & \sum_{i \in V} \sum_{j \in V} s_{ij} y_{ij} \\ \text{subject to} \quad & \sum_{j \in V} y_{ij} = 1, \quad \forall i \in V, \\ & \sum_{j \in V} x_j = \bar{K}, \quad x_j \in \{0, 1\}, \\ & 0 \le y_{ij} \le x_j, \quad \forall i, j \in V. \end{aligned} \tag{3}$$

Here, $x_j \in \{0, 1\}$ indicates whether row j is retained, while $y_{ij} \in \{0, 1\}$ indicates whether row j is selected as the representative of original row i. For example, if row 5 is retained and selected as the representative of row 3, then $y_{3,5} = 1$. The first constraint ensures that every original row is assigned to exactly one representative. The second constraint requires exactly $\bar{K}$ rows to be retained, while

the third ensures that only retained rows can serve as representatives. The objective maximizes the total similarity between original rows and their assigned representatives. However, the contribution of each retained row to the overall coverage depends on which other rows are selected, making it challenging to identify the best subset and motivating the greedy approximation discussed next.

4.3.2 Greedy Optimization. Finding an optimal solution to the above coverage maximization problem is NP-hard in general, which is too costly. Thus, we adopt a greedy approximation method as the solution. Let $\mathcal{A}$ denote the set of rows selected so far. For any candidate row $j \in V \setminus \mathcal{A}$, its marginal coverage gain is defined as $\Delta(j \mid \mathcal{A}) = F_{\text{cov}}(\mathcal{A} \cup \{j\}) - F_{\text{cov}}(\mathcal{A})$. Starting from an empty set, the procedure repeatedly selects the row maximizing this gain to construct a priority ordering, as illustrated in Algorithm 2.

Algorithm 2: Greedy Row Ranking

Input: Row indices $V = \{1, \ldots, n\}$, similarity matrix S, row budget $\bar{K}$
Output: Priority ordering π_T, retained set $\mathcal{A}_{\bar{K}}$
$\mathcal{A} \leftarrow \emptyset, \quad \pi_T \leftarrow ()$
for each $i \in V$ **do**
  $m_i \leftarrow 0$
for $t = 1, \ldots, n$ **do**
  $j^\star \leftarrow \operatorname{argmax}_{j \in V \setminus \mathcal{A}} \sum_{i \in V} \max(0,\, s_{ij} - m_i)$
  $\pi_T \leftarrow (\pi_T, j^\star)$
  $\mathcal{A} \leftarrow \mathcal{A} \cup \{j^\star\}$
  for each $i \in V$ **do**
    $m_i \leftarrow \max(m_i, s_{ij^\star})$
$\mathcal{A}_{\bar{K}} \leftarrow \{\pi_1, \ldots, \pi_{\bar{K}}\}$
return $\pi_T, \text{sort}(\mathcal{A}_{\bar{K}})$

Algorithm 2 works as follows: starting from an empty set $\mathcal{A}$, it iteratively selects the row that provides the largest marginal coverage gain and returns a set of rows $\mathcal{A}_{\bar{K}}$ to retain. To efficiently evaluate candidate rows, it maintains m_i, the maximum similarity between each original row i and the rows selected so far. The marginal gain of a candidate j can then be computed as $\Delta(j \mid \mathcal{A}) = \sum_{i \in V} \max(0, s_{ij} - m_i)$. Each selected row is appended to the priority ordering π_T, and the maintained similarities are updated accordingly. The first $\bar{K}$ entries determine the retained row set. The results include the complete ranking, allowing different row budgets to be applied without recomputing the selection order.

Approximation error bound Theorem 4.2 establishes the properties of our coverage objective and the approximation guarantee of our greedy selection. The $(1 - 1/e)$ bound indicated in the theorem guarantees that greedy selection achieves at least this fraction of the optimal coverage in above truncation problem.

Theorem 4.2 (Greedy Coverage Guarantee). *Given a nonnegative similarity matrix $S = [s_{ij}]$, the coverage function F_{cov} defined in Equation 2 is normalized, monotone, and submodular. Let $\mathcal{A}^\star$ denote the optimal retained set of size $\bar{K} \geq 1$, and let $\mathcal{A}_{\bar{K}}$ be the set obtained after $\bar{K}$ greedy iterations. Then,*

$$F_{\text{cov}}(\mathcal{A}_{\bar{K}}) \geq \left[1 - \left(1 - \frac{1}{\bar{K}}\right)^{\bar{K}}\right] F_{\text{cov}}(\mathcal{A}^\star) \geq (1 - 1/e) F_{\text{cov}}(\mathcal{A}^\star). \quad (4)$$

Proof. For each row i, define $m_i(\mathcal{A}) = \max_{j \in \mathcal{A}} s_{ij}$, with $m_i(\emptyset) = 0$. By Equation 2, we have $F_{\text{cov}}(\mathcal{A}) = \sum_{i \in V} m_i(\mathcal{A})$, and hence $F_{\text{cov}}(\emptyset) = 0$, establishing normalization. Next, since $s_{ij} \geq 0$, for any $\mathcal{A} \subseteq \mathcal{B} \subseteq V$, we have $m_i(\mathcal{A}) \leq m_i(\mathcal{B})$ for every i. Summing over all $i \in V$ yields $F_{\text{cov}}(\mathcal{A}) \leq F_{\text{cov}}(\mathcal{B})$, establishing monotonicity.

To establish submodularity, consider $\mathcal{A} \subseteq \mathcal{B}$ and $x \notin \mathcal{B}$. The marginal gain of adding x is $\Delta(x \mid \mathcal{A}) = \sum_{i \in V} \max(0, s_{ix} - m_i(\mathcal{A}))$. Since $m_i(\mathcal{A}) \leq m_i(\mathcal{B})$, it follows that $\Delta(x \mid \mathcal{A}) \geq \Delta(x \mid \mathcal{B})$, satisfying the diminishing-returns property of submodularity.

Since F_{cov} is normalized, monotone, and submodular, and Algorithm 2 selects the row with the largest marginal gain at each iteration, the classical greedy approximation guarantee for monotone submodular maximization under a cardinality constraint [47] applies directly, yielding the stated bound. □

Complexity With the $n \times n$ similarity matrix precomputed, the marginal gain of each candidate can be evaluated in $O(n)$ time using the maintained maximum similarities. Each iteration evaluates at most n candidates, requiring $O(n^2)$ time. Consequently, selecting the first $\bar{K}$ rows takes $O(\bar{K}n^2)$ time, while constructing the complete priority ordering takes $O(n^3)$ time. However, finding the optimal retained subset by exhaustive search includes $\binom{n}{\bar{K}}$ candidate subsets. Since evaluating $F_{\text{cov}}(\mathcal{A})$ for a subset of size $\bar{K}$ takes $O(n\bar{K})$ time, exhaustive enumeration requires $O\left(\binom{n}{\bar{K}} n\bar{K}\right)$ time.

4.3.3 Discussion. Finally, we illustrate the potential generality of above approach to a broader scope of table understanding tasks beyond CTA and CPA. To accommodate a new table understanding task, users just need to specify the permissible truncation operations through the task-level policy, G, and identify the protected table regions through instance-level metadata, M. Then the task's information-preservation requirements can be expressed through the protected row and column sets, P_r and P_c. This allows TSLoRA to be adapted for a wide range of data management tasks; however, since this is a data management specific system, it will not help for non-data management tasks.

Beyond row-wise truncation, our framework also supports column-wise selection through two strategies. The `nearest` strategy prioritizes columns based on their positional proximity to task-relevant regions specified by the metadata, while the `coverage` strategy selects representative columns based on content similarity, following the same representative-coverage principle used for row selection. Both strategies preserve the original ordering of retained columns and operate within the task-dependent truncation constraints.

5 OPTIMIZING FINE-TUNING FOR TABLE UNDERSTANDING

In spite of the effectiveness of vanilla LoRA in fine-tuning LLMs in general, there is still room for improvement in adopting it to table understanding tasks, since data management tasks have specific structure and need. Since the backbone LLMs are usually pretrained on large corpora of general documents with large amounts of unstructured texts, it has been widely recognized in previous studies that some models have certain limitations in understanding the structural information of tables. We hope to avoid the overhead that previous studies [23, 39, 63] encountered as they addressed such problems with a costly, full fine-tuning process. To this end, we proposed a light-weight two-stage strategy as a solution.

5.1 Overview of TSLoRA's Two-Stage Strategy

Table understanding tasks require models to combine information across cells while capturing their structural relationships within rows and columns. To support such reasoning efficiently, our two-stage strategy focuses on adapting how information is collected across table tokens and transformed within the LLMs. The models considered in this work are based on the Transformer architecture [57], whose layers primarily consist of a self-attention component followed by a multi-layer perceptron (MLP) component. Self-attention enables each token to gather information from other tokens. Specifically, the query, key, and value projection matrices transform each token's representation into a query, key, and value vector, respectively. For a given token, its query vector is compared with the key vectors of tokens in the sequence to compute attention weights. These weights determine a weighted sum of the corresponding value vectors. Let Q, K, and V denote the matrices formed by stacking the query, key, and value vectors of the tokens, respectively. The scaled dot-product attention operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V, \tag{5}$$

where d_k is the dimension of the query and key vectors. In decoder-only LLMs, a causal mask additionally restricts the positions each token can attend to. The output projection transforms the aggregated information, after which the MLP processes each token's updated representation individually, without directly exchanging information between tokens. LoRA can be applied to weight matrices in both components by representing their trainable updates as the product BA as described in Section 2.2.

Our two-stage strategy uses these projections for different learning objectives. In the first stage, we apply LoRA only to the key and value projections, allowing supervision on cell-pair relationships to adjust attention weights and the information aggregated from table tokens (Section 5.2). In the second stage, we transfer the structurally adapted LoRA parameters and extend adaptation to the other attention and MLP projections for downstream tasks. We further introduce target and context role information into the key and value projections to distinguish task-specified substructures from the surrounding table context (Section 5.3).

5.2 Structure-aware Supervision Training for Table Understanding

In the first stage, we aim to equip the LoRA adapters with the capability to capture and manipulate the unique structural relationships within tables, i.e. the relative positions within the table in the input training instances. Rather than relying on downstream table understanding task ground truth to convey table structure indirectly, we introduce a structure-derived supervision task that explicitly trains the LoRA adapters to recognize positional relationships between pairs of table cells. This allows TSLoRA to better solve table understanding tasks than LLM only approaches.

We can directly construct the table understanding training instances for this stage from the tables obtained through the training data preparation, as follows: To begin with, let $T = (t_{ij})$ be a table whose cells have known row and column coordinates. We sample two *distinct* cells $a = (i, j)$ and $b = (i', j')$, and use their coordinates to derive a ground-truth relationship label, $y(a, b)$, indicating whether they belong to the same row, the same column, or neither.

Restricting the sampled pairs to distinct cells makes the three relationship labels mutually exclusive, since a cell paired with itself would satisfy both the same-row and same-column conditions. Moreover, because each label is determined directly from the cells' row and column coordinates, we can obtain the supervision signals for a simple three-label prediction task automatically without additional manual annotations.

We next consider the ordering of the serialized table and natural-language instruction within each training instance. To enable the LLMs to better understand the positional information, we deliberately put the serialized table before the natural-language instruction rather than the standard practice of putting the instruction before the table. This aligns with the observation in the recent study [48] that such an ordering improves the effectiveness of similar tasks. The resulting training sequence, including both the input prompt and its expected answer, is organized as follows:

$$X_{\text{struct}} = [\, X_{\text{table}};\ X_{\text{instruction}}(a, b);\ X_{\text{answer}}(\text{question}) \,]. \tag{6}$$

The rationale for above practice lies in the causal self-attention mechanism in Transformers [57]: the hidden state at position p can attend only to positions $q \le p$. A decoder-only LLM adopts this causal attention design. With a "table-first" serialization, every instruction token follows the table segment. Consequently, table-token contextual representations cannot attend to the subsequent task instructions in the forward pass. If the input were instead $[X_{\text{instruction}}; X_{\text{table}}; \ldots]$, each table token could attend to the instruction prefix, and the resulting table-token states would mix structural information with instruction context. This design enables learning table-token representations without conditioning on the specific cell-pair instruction, while the subsequent prediction can still incorporate both the table content and the instruction.

After obtaining the derived training instances, we introduce the training process of the first stage: to reduce overhead while incorporating structural information, we apply LoRA only to the key and value matrices of the self-attention component as described in Section 5.1, restricted to table tokens so that this adaptation remains instruction-independent. The motivation is to learn table-structural representations separately from downstream task-specific adaptation, avoiding potential interference from different task instructions between two stages. This allows us to better solve table understanding tasks. Formally, let θ denote the frozen backbone and $\phi^{(1)}$ denote the LoRA parameters on these selected matrices. Let $\mathcal{D}_{\text{struct}}$ denote the structure-stage training dataset consisting of examples (x, y), where x contains the serialized table followed by the cell-pair instruction, and y is the ground-truth structural relationship label. The supervised generation loss is defined as follows:

$$\mathcal{L}_{\text{struct}}(\phi^{(1)}) = -\sum_{(x,y)\in\mathcal{D}_{\text{struct}}} \log p_{\theta,\phi^{(1)}}(y \mid x). \tag{7}$$

5.3 Substructure Alignment for Table Understanding

While recognizing structural relationships among table cells is important, table understanding sometimes may also focus on specific sub-tables relevant to a given instruction. Thus, the structural knowledge learned in the first stage needs to be further adapted to distinguish task-relevant substructures from the surrounding table context. To achieve this, we transfer the structurally adapted key and value from LoRA adapters that were built in the first stage and continue fine-tuning for downstream tasks over both the attention and the MLP components. In the second stage, the model should not only understand the structure of the input table but also distinguish the target specified by the task from the surrounding table context. Meanwhile, the first stage does not explicitly supervise this distinction, as its training labels encode only positional relationships between sampled table cells. We reach this goal in this stage by modifying the trainable parameters in vanilla LoRA to encode the alignment between the instruction and substructures in the table that are essential to answer the question. Specifically, we introduce a lightweight role-conditioned adjustment to vanilla LoRA to facilitate the alignment between task instructions and relevant table substructures. The key intuition is that different parts of a table play different roles in answering a given question: the target sub-structure provides the primary information to be examined, while the rest of the table serves as supporting context.

Building on the LoRA formulation, we introduce two learnable role vectors, $\mathbf{e}_{\text{tar}}, \mathbf{e}_{\text{ctx}} \in \mathbb{R}^r$, to indicate the target substructure and its surrounding table context respectively, which only add negligible trainable parameter overhead. During the second stage, these vectors are jointly optimized with the LoRA parameters using downstream task supervision. Specifically, for a token t, the role vector $\mathbf{e}_t$ is set to $\mathbf{e}_{\text{tar}}$ if the token belongs to the target substructure, $\mathbf{e}_{\text{ctx}}$ if it belongs to the remaining table context, and $\mathbf{0}$ if it is outside the table. We add $\mathbf{e}_t$ to the low-rank representation $A\mathbf{h}_t$ before multiplying with matrix B from key and value matrices only, where $\mathbf{h}_t \in \mathbb{R}^d$ denotes the hidden state of a token t. Thus, the LoRA update $(\frac{\alpha}{r})BA\mathbf{h}_t$ becomes $\frac{\alpha}{r}B(A\mathbf{h}_t + \mathbf{e}_t)$.

Consider the CTA example in Figure 4, where the task specifies `col3` as the target. The tokens corresponding to cells in this column, such as *Still Not a Player* and *Mary Jane*, receive the target role vector $\mathbf{e}_{\text{tar}}$, while tokens from the remaining columns receive the context role vector $\mathbf{e}_{\text{ctx}}$. The ground-truth annotation for the target column is the DBpedia class `Single`. This example illustrates how the task-specified target is explicitly distinguished from the surrounding table context through role-conditioned LoRA adaptation.

Finally, we conduct the training of the second stage based on the following objective: let $\phi^{(2)}$ denote the LoRA parameters in this stage, with key and value matrices transferred from the structurally adapted parameters $\phi^{(1)}$ obtained in the first stage, and let ψ denote the collection of learnable target and context role vectors. Given a downstream training dataset $\mathcal{D}_{\text{task}}$, the optimization objective is

$$\mathcal{L}_{\text{task}}(\phi^{(2)}, \psi) = -\sum_{(x,y)\in\mathcal{D}_{\text{task}}} \log p_{\theta,\phi^{(2)},\psi}(y \mid x), \tag{8}$$

where x consists of the serialized table and task-specific instruction, y denotes the expected output, and θ represents the frozen backbone. Both $\phi^{(2)}$ and ψ are updated through the same supervision, without introducing an additional alignment-specific loss.

6 EVALUATION

6.1 Experiment Setup

Source corpora **Statistic**	**WikiTables**	**WDC**
Tables	1,652,771	4,191,408
Rows/table (min/mean/med./max)	0/10.91/5/4,670	1/69.7/4/391,572
Columns/table (min/mean/med./max)	0/4.96/4/398	0/4.58/4/124
Evaluation Benchmark **Statistic**	**MMTU**	**SOTAB Test**
CTA instances	881	15,040
CTA distinct labels	93	91
CPA instances	873	23,156
CPA distinct labels	157	176
Rows/table (min/mean/med./max)	5/63.9/46/197	10/224.1/40/30,569
Columns/table (min/mean/med./max)	4/4.37/4/8	3/8.80/8/30

Table 1: Statistics of source corpora, and evaluation benchmarks.

6.1.1 Datasets and LLMs. To compare our table understanding-specific solution with existing methods, we first conducted LoRA based fine-tuning on two large-scale table corpora, WikiTables [1] and WDC [2]. As discussed earlier, while our methods work for a wide range of table understanding tasks, our evaluation concentrated on two specific table understanding tasks: Column Type Annotation (CTA) and Column Property Annotation (CPA). Since our methodology primarily focuses on fine-tuning LLMs, we chose datasets that are focused on evaluating LLM-based solutions as suggested by recent studies [20, 62]. We thus did not consider the earlier benchmarking tasks created from similar table corpora such as [12, 28]. Specifically, the CTA/CPA tasks over WikiTables are from [32], which is also included in the Massive Multi-Task Table Understanding and Reasoning Benchmark (MMTU) [62] for LLMs. The CTA/CPA tasks over WDC are from the SOTAB [36] benchmark. We only use their test sets for evaluation since their training sets are constructed to train a PLM-based table encoder [17, 45, 53] rather than fine-tuning general purposed LLMs. Table 1 summarizes the characteristics of the two corpora and their associated CTA/CPA evaluation sets. The two benchmarks differ considerably in corpus scale and table characteristics, allowing us to study LoRA-based adaptation across heterogeneous tabular data. In addition, we also show that our proposed framework can support other table related tasks from the MMTU [62] in Section 6.4.

For model fine-tuning, we adopted two publicly available open-weight models, Llama-3.2-3B-Instruct [44] and Qwen3-4B [52], with approximately 3 and 4 billion parameters respectively. These two state-of-the-art LLMs represent two different LLM families at comparable parameter scales, allowing us to investigate the effectiveness of our proposed techniques across different architectures.

6.1.2 Evaluation Metrics. Because TSLoRA is designed to keep effectiveness high while reducing costs, our experiments centered on seeing if TSLoRA could reduce costs without substantially decreasing effectiveness. Since both CTA and CPA are multi-class

[1] http://websail-fe.cs.northwestern.edu/TabEL/
[2] https://webdatacommons.org/structureddata/schemaorgtables/

classification tasks, we use both Micro F_1 and Macro F_1 scores as the evaluation metrics for effectiveness. This follows the route of previous studies [45, 53, 61]. Specifically, the Macro F_1 score is calculated by averaging the F_1 score from all classes.

6.1.3 Baseline Methods. To demonstrate the effectiveness of our fine-tuning techniques, we primarily compare the performance of a prompt-only approach without fine-tuning as well as the full fine-tuning method on all tasks. In addition, we also compare our proposed TSLoRA framework with state-of-the-art methods that fine-tune PLMs [17, 45, 53] and prompt over much larger closed-source LLMs such as GPT [20, 33, 61]. Specifically, among LLM based methods CHORUS [33] designed universal prompt strategies for multiple table related tasks while CENTS [61] also cared about reducing the length of contexts in prompt input, both of which have similar objectives with our proposed methods. Additionally, we conducted further ablation studies in Section 6.3 to show the effect of proposed techniques by comparing with the baseline of fine-tuning with vanilla LoRA [27].

6.1.4 Environment and Settings. All code was written in Python. All experiments were conducted on the Fir and Nibi clusters from the Digital Research Alliance of Canada [14, 15]. Both clusters employ SLURM [67] for job scheduling. Each experiment was allocated 16 GB of system memory and performed on a 40 GB Multi-Instance GPU (MIG) partition of an NVIDIA H100 80 GB GPU.

For Llama-3.2-3B-Instruct and Qwen3-4B, we set the per-device micro-batch sizes to 4 and 2, respectively, with gradient accumulation steps of 2 and 4, yielding an effective batch size of 8 for both models. This configuration accommodates different GPU memory requirements of the two models while maintaining a consistent effective batch size for comparison. We set the maximum length to 1,024 tokens for WikiTables and 2,048 tokens for WDC for each training instance. We used a learning rate of 1×10^{-4} for LoRA fine-tuning and 1×10^{-5} for full-model fine-tuning. Across all experiments, we employed PyTorch's scaled dot-product attention (SDPA) implementation [3]. To evaluate adaptation under controlled supervision data budgets, each specified training budget was split equally between the CTA and CPA tasks (e.g., a budget of 6,000 instances comprises 3,000 CTA and 3,000 CPA instances).

6.2 Main Results

We report the results of CTA and CPA over the test sets of MMTU and SOTAB, respectively. The training instances were created via the techniques developed in Section 4 from the corresponding table corpora WikiTables and WDC. The training budgets are set as 6000. We directly cite the results of the baselines from [62] for MMTU, and from recent studies [61] for SOTAB, respectively.

In this first set of experiments, we checked to see if, as hoped, the performance of TSLoRA can be closer to that of the much more expensive full fine-tuning approaches as well as much better than that of the no tuning approaches. Across both MMTU and SOTAB, TSLoRA substantially improves over prompting the same Llama-3.2-3B [44] and Qwen3-4B [52] backbones on both CTA and CPA. Compared with full supervised fine-tuning, TSLoRA remains competitive while updating only LoRA parameters and using fewer training instances. To ensure the effectiveness of full fine-tuning approach and avoid underfitting, we set the training budget of these approaches as 10,000. Despite this smaller training budget (6000 by default), TSLoRA achieves performance comparable to Full FT across both benchmarks and, with Qwen3-4B, exceeds its Full FT counterpart on several metrics. Such results show that TSLoRA achieves good effectiveness while being substantially cheaper.

Next we compare with existing methods based on prompting LLMs. As shown in Table 2, the adapted 3B–4B models also outperform prompt based methods on substantially larger models, including the frontier GPT5 [50] and DeepSeek-V3/R1 [10, 11] on MMTU,

[3] https://pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html

	CTA		CPA	
Method	**Macro**	**Micro**	**Macro**	**Micro**
LLM (Prompt-Only)				
GPT5-Chat	–	55.40	–	29.60
GPT5-Reasoning	–	46.20	–	49.20
DeepSeek-V3	–	40.90	–	25.80
DeepSeek-R1	–	37.10	–	28.00
Llama-3.2-3B	16.85	54.94	0.23	1.83
Qwen3-4B	20.91	65.61	2.85	1.83
LLM (Full Fine-Tuning)				
Llama-3.2-3B (Full FT)	76.65	95.01	67.58	69.42
Qwen3-4B (Full FT)	69.32	92.28	59.60	69.06
LLM (TSLoRA)				
TSLoRA (Llama-3.2-3B)	**76.58**	**92.51**	**64.57**	**67.81**
TSLoRA (Qwen3-4B)	72.49	92.17	62.11	66.55

Table 2: Effectiveness comparison on the MMTU benchmark (WikiTables corpus). The best results among prompt-only and TSLoRA models are bolded; overall best results are underlined. A dash (–) indicates that the corresponding metric was not reported in the original baseline.

	CTA		CPA	
Method	**Macro**	**Micro**	**Macro**	**Micro**
LLM (Prompt-Only)				
Qwen-Plus	48.82	52.61	45.41	53.48
Llama-3.2-3B	4.60	5.84	2.70	3.96
Qwen3-4B	26.24	30.94	11.84	13.97
Prompt LLM: SOTA methods				
ArcheType	–	65.20	–	–
CHORUS	–	59.40	–	–
CENTS	–	80.40	–	**79.10**
LLM (Full Fine-Tuning)				
Llama-3.2-3B	83.39	86.24	71.67	76.06
Qwen3-4B	84.36	86.36	**76.71**	77.25
Fine-tuning PLMs: SOTA methods				
RECA	68.21	68.52	47.56	54.31
DODUO	78.51	81.32	74.95	78.85
Watchog	84.19	86.23	72.60	76.52
REVEAL+	88.10	88.74	78.15	80.81
LLM (TSLoRA)				
TSLoRA (Llama-3.2-3B)	83.03	84.90	69.17	73.83
TSLoRA (Qwen3-4B)	**85.07**	**86.47**	72.98	77.64

Table 3: Effectiveness comparison on the SOTAB benchmark (WDC corpus). Best LLM results are in bold; overall best results including PLMs are underlined. A dash (–) indicates that the corresponding metric was not reported in the original baseline.

as well as GPT-3.5-Turbo and Qwen-Plus [65] on SOTAB. These results indicate that increasing model scale alone does not substitute for adaptation to the target table-understanding tasks. In addition, the results of state-of-the-art prompt based methods [20, 33, 61] in Table 3 provide another interesting comparison. We can see that TSLoRA achieves comparable performance over much smaller open sourced models (Llama-3.2-3B and Qwen3-4B) while such methods were using GPT. For example, TSLoRA achieves CTA Micro F_1 scores of 84.90 and 86.47 with Llama-3.2-3B and Qwen3-4B, respectively. Besides, these methods are evaluated on simplified SOTAB benchmarks with 27 CTA and 41 relation labels, whereas our evaluation retains the original 91 CTA and 176 CPA labels. Unlike these prompt based methods, TSLoRA could benefit from the fine-tuning process and thus adapt to the finer-grained label space.

Finally, we compare TSLoRA against state-of-the-art PLM-based solutions (Table 3). These methods are fully fine-tuned using 64,884 CTA and 74,216 CPA instances from the training set of the original datasets, whereas TSLoRA only needs 3,000 instances that can be automatically generated from the table corpora without human annotationfor each task. Despite using less than 5% of their task-specific supervision, TSLoRA outperforms RECA [55] and Watchog [45] across both CTA and CPA and is only worse than REVEAL+ [17]. However, REVEAL+ still needed to perform the fine-tuning process for each task separately as other methods in the same category did, while TSLoRA just needs one fine-tuning process to support both tasks. Overall, these comparisons show that adapting a small general-purpose LLM with TSLoRA can achieve competitive performance under a substantially smaller supervision budget.

6.3 TSLoRA Component and Settings Analysis

We next conduct an in-depth analysis to systematically evaluate our proposed techniques. Specifically, we investigate three central questions: (i) What are the individual contributions and interplay of token-efficient table truncation technique (denoted as TabTrunc) in Section 4 and table structure learning as well as two-stage training strategy (denoted as StructAware) in Section 5? (Section 6.3.1) (ii) Do the performance advantages of TSLoRA hold across diverse supervision scales, or are they an artifact of the default budget? (Section 6.3.2) (iii) Does input token reduction yield measurable downstream speedups during fine-tuning? (Section 6.3.3)

		CTA		CPA	
Backbone	**Variant**	**Macro**	**Micro**	**Macro**	**Micro**
Llama-3.2-3B	Vanilla LoRA	70.01	93.30	63.78	66.67
	+ TabTrunc	74.67	92.17	63.03	68.73
	+ StructAware	75.70	**93.98**	**65.60**	**70.68**
	TSLoRA (Full)	**76.58**	92.51	64.57	67.81
Qwen3-4B	Vanilla LoRA	68.51	92.05	61.05	64.95
	+ TabTrunc	69.61	92.17	60.75	64.83
	+ StructAware	**77.41**	**92.40**	**65.23**	65.61
	TSLoRA (Full)	72.49	92.17	62.11	**66.55**

Table 4: Ablation study of TSLoRA on MMTU under the 6,000-instance training budget. Best results per backbone are in bold.

6.3.1 Component Ablation. We first report the results of ablation studies to test the effect of the two major proposed techniques. In Tables 4 and 5 we isolate the individual contributions of our two core stages under the default training budget. The results highlight a clear, complementary division of roles:

		CTA		CPA	
Backbone	**Variant**	**Macro**	**Micro**	**Macro**	**Micro**
Llama-3.2-3B	Vanilla LoRA	81.99	83.93	65.50	72.04
	+ TabTrunc	83.28	84.77	68.86	73.88
	+ StructAware	**84.41**	**85.56**	**69.90**	**74.00**
	TSLoRA (Full)	83.03	84.90	69.17	73.83
Qwen3-4B	Vanilla LoRA	83.18	84.81	69.80	75.12
	+ TabTrunc	84.72	86.06	70.92	76.35
	+ StructAware	**85.42**	**87.07**	**73.35**	**77.75**
	TSLoRA (Full)	85.07	86.47	72.98	77.64

Table 5: Ablation study of TSLoRA on SOTAB under the 6,000-instance training budget. Best results per backbone are in bold.

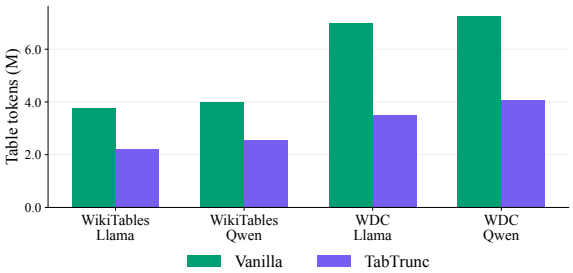


Figure 5: Comparison of table tokens processed by vanilla LoRA and TabTrunc under the 6,000-instance training budget.

StructAware provides the primary representation gains. The two-stage training strategy adds the additional information that TSLoRA uses to improve effectiveness. Therefore, the goal in testing this component was to see if it alone was responsible for most of TSLoRA's effectiveness; this is justified by the experimental results. Specifically, StructAware consistently yields peak or near-peak effectiveness across all tasks and models. On Qwen3-4B, StructAware improves MMTU CTA Macro F_1 from 68.51 to 77.41 and CPA Macro F_1 from 61.05 to 65.23. On Llama-3.2-3B, it improves SOTAB CPA Macro F_1 from 65.50 to 69.90. Because standard LoRA initializes adapter matrices randomly or to zero, it remains agnostic to tabular geometry; StructAware overcomes this by equipping the adapter weights with an explicit cell-pair relational prior (distinguishing same-row, same-column, and unrelated cells), plus a task-aware substructure alignment in the two-stage training process.

TabTrunc preserves accuracy while discarding redundant inputs. The goal of TabTrunc is to reduce costs by including a smaller set of tokens in training instances, which will then save the cost of downstream tasks. As shown in Figure 5, TabTrunc reduces table tokens by 1.44–1.56 million (35.94%–41.28%) on WikiTables and 3.21–3.46 million (44.17%–49.54%) on WDC across the two backbones. The results in Table 4 and 5 reveal that TabTrunc does not necessarily degrade task performance. In fact, TabTrunc matches or even slightly outperforms Vanilla LoRA on 12 out of the 16 evaluated metrics. For example, on Llama-3.2-3B the Macro F_1 is improved by 4.66 on CTA of MMTU and by 3.36 on SOTAB CPA. This shows that raw serialized tables sometimes contain redundant

or noisy records that dilute self-attention; filtering them preserves semantic representation while drastically reducing the input tokens.

Synergy in TSLoRA. The full TSLoRA framework combines both components, non-trivially outperforming Vanilla LoRA across backbones (e.g., improve Macro F_1 by 6.57 on Llama MMTU CTA and 3.18 on Qwen SOTAB CPA, respectively). While StructAware alone achieves marginally higher peak metrics in unconstrained settings, it processes full serialized tables. TSLoRA retains the overwhelming majority of StructAware's structural gains while unlocking the substantial computational and memory savings enabled by TabTrunc.

6.3.2 Robustness across Training Budgets. Next we report the mean Macro F_1 across budgets spanning 2,000 to 15,000 training instances. As shown in Figure 6, TSLoRA maintains a persistent, stable margin over Vanilla LoRA across the entire evaluation spectrum on both the MMTU and SOTAB datasets. The scaling curves exhibit smooth, monotonic progress for both methods as supervision increases, with TSLoRA consistently outperforming the Vanilla LoRA baseline. Crucially, the relative performance advantage observed at 6,000 instances is representative of the entire trajectory: TSLoRA neither degrades under constrained budgets nor exhibits diminishing returns as data expands to 15,000 instances. This confirms that the empirical benefits of TSLoRA scale predictably and hold across diverse supervision levels.

6.3.3 Fine-tuning Time and Token Efficiency. Finally, we examine whether the sequence truncation introduced by TabTrunc yields practical computational savings during downstream adaptation. While LoRA reduces parameter and memory overhead by freezing the model backbone, standard PEFT still forces the frozen underlying models to process every serialized table token across all training steps. TabTrunc directly addresses this token inefficiency by pruning redundant table rows prior to tokenization.

Across our experiments under the default training budget on MMTU and SOTAB with both backbones, TabTrunc can shorten downstream fine-tuning from an average of 148.6 minutes to 114.5 minutes, achieving a consistent 23.0% reduction in training time. This speedup directly reflects the reduced sequence lengths achieved by TabTrunc. In practice, serialized table contents account for the bulk of input tokens during training. By pruning redundant rows prior to tokenization, TabTrunc shortens the effective sequence length for each instance. Because self-attention and activation memory scale with sequence length, this input reduction yields concrete per-step throughput improvements during both forward and backward passes. These results confirm that TSLoRA provides practical training acceleration alongside parameter efficiency.

6.4 Case Study

Our main evaluation focuses on CTA and CPA over lightweight backbones as controlled settings for studying the effect of LoRA adaptation to tabular data. In this section, we present three case studies that go beyond this setting. First, we apply TSLoRA to a larger backbone to examine whether its benefits carry over as model size grows. Second, we evaluate on other table-related tasks to demonstrate the capacity of TSLoRA in generalization. Finally, we perform a micro-bench study to show that LoRA adaptation enables the model to make more effective use of surrounding table

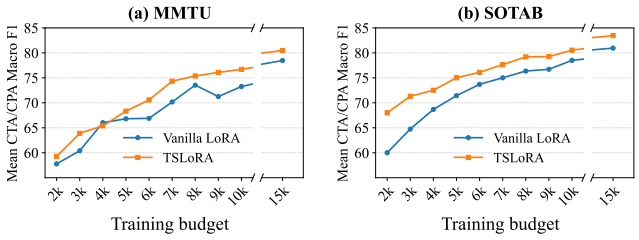


Figure 6: Mean CTA/CPA Macro F_1 score of Llama-3.2-3B-Instruct across varying training budgets on (a) MMTU and (b) SOTAB.

context, providing further evidence that task-specific adaptation brings additional benefits to models when processing tabular inputs.

	CTA		CPA	
Variant	**Macro**	**Micro**	**Macro**	**Micro**
Prompt-only	16.07	74.80	2.87	11.80
Vanilla LoRA	80.50	94.67	68.20	72.28
+ TabTrunc	78.16	94.55	70.19	73.77
+ StructAware	83.16	95.01	**71.17**	**74.23**
TSLoRA (Full)	**84.07**	**95.57**	69.25	72.05

Table 6: Results of TSLoRA and its variants with Llama-3.1-8B-Instruct on MMTU under a 6,000-instance training budget. Best results among the tuned variants are in bold.

6.4.1 Scaling to a Larger Backbone. To examine the benefits that TSLoRA can carry over to larger models, we apply TSLoRA and its variants to Llama-3.1-8B-Instruct [43] (Table 6). We see that TSLoRA can substantially improve the overall performance and remains effective on larger backbones. For example, the 8B model reaches only 16.07 CTA and 2.87 CPA Macro F_1 without adaptation, remaining far from usable. TSLoRA improves the CTA Macro F_1 to 84.07 and achieves the best CTA results among all variants.

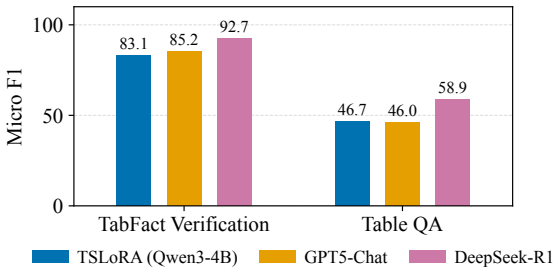


Figure 7: Performance on MMTU TabFact verification and Table QA. GPT5-Chat and DeepSeek-R1 results are reported by MMTU.

6.4.2 Support Tasks Beyond CTA and CPA. We applied TSLoRA to Qwen3-4B [52] over two additional MMTU tasks: TabFact verification, which originated from the TabFact benchmark [6], and Table QA, which originated from four different benchmarks [8, 9, 51, 60]. Table QA answers natural-language questions based on the information contained in a table, while TabFact Verification determines whether a given statement is supported by the table evidence or not.

For each of these two tasks, we involved an additional set of 10,000 training instances for fine-tuning. We compare against the prompt based baselines over GPT5-Chat and DeepSeek-R1 results reported by MMTU in Figure 7. On TabFact, TSLoRA reaches a Micro F_1 of 83.1, approaching GPT5-Chat at 85.2, while on Table QA it reaches 46.7, slightly exceeding GPT5-Chat at 46.0. These results show that the same lightweight adaptation framework can be easily extended to qualitatively different table related tasks. Meanwhile, although there is still performance gap over the reasoning-intensive table tasks, it is mainly caused by the limitations of smaller open sourced LLMs in reasoning capability when comparing with frontier closed-source models. TSLoRA can still provide a practical mechanism for extending small open-source models to complicated tabular understanding tasks and can already reach performance competitive with some much larger closed-source models.

6.4.3 Table Context Utilization Learning. In tabular data, the surrounding context, including but not limited to co-occurring values and adjacent columns, is essential to capture relational dependencies and resolving ambiguous values. While human analysts and specialized table representations inherently rely on this multi-dimensional structure to interpret tabular inputs, LLMs often struggle to process it effectively out of the box, frequently treating context information as distracting noise.

Figure 8 illustrates such a challenge and examines whether parameter-efficient adaptation can overcome it. Here, we define **context** as the non-target columns in the table: for CTA, this includes all columns alongside the candidate target column; for CPA, the context comprises of all remaining columns outside the target column pair. Specifically, on pre-trained backbones, providing context columns consistently degrades model accuracy across both architectures. On CTA, excluding surrounding context improves the Macro F_1 from 16.9% to 17.8% for Llama and from 20.9% to 23.8% for Qwen, while CPA scores remain near floor levels ($< 5\%$) and likewise decline when context is retained. This confirms that without task-specific alignment, pre-trained LLMs suffer from context distraction, as general-purpose self-attention fails to separate informative signals from noise among surrounding tabular tokens.

At the same time, lightweight LoRA adaptation completely reverses this failure mode. When adapters are trained with a modest budget of 2,000 instances with versus without context columns (and evaluated on standard context-preserving MMTU test sets), the surrounding table context transforms from a distractor into a major performance driver. With context retained, CTA Macro F_1 surges from 48.2% to 68.8% on Llama and from 47.9% to 64.5% on Qwen, respectively. The effect is particularly pronounced for CPA, where relational reasoning across columns is crucial: context yields Macro F_1 gains of 25.4 on Llama and 11.3 on Qwen, respectively. These consistent gains across model families demonstrate that LoRA adaptation does more than map inputs to target label spaces; it equips the attention mechanism to navigate, filter, and exploit structural table context that original models discard as noise.

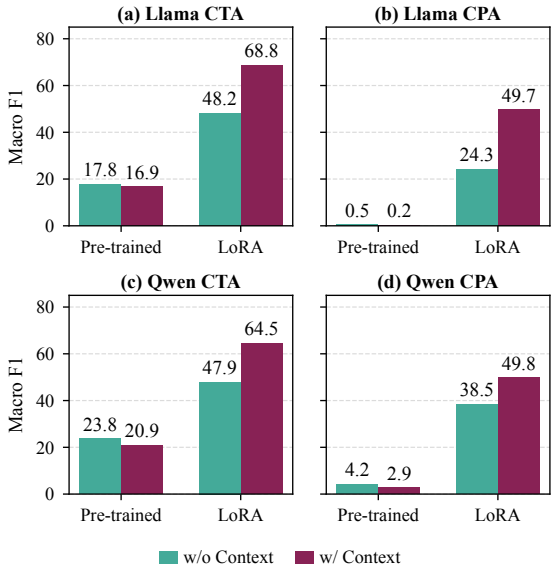


Figure 8: Effect of context columns on pre-trained and LoRA-adapted backbones for MMTU CTA and CPA.

7 RELATED WORK

7.1 Table Understanding

Table understanding has been an important data management topic for decades. Table annotation tasks were first introduced in [41]. Early studies such as Sherlock [30] and SATO [68] employed manual feature engineering over tables and used classic machine learning models for prediction. Later studies mainly relied on fine-tuning PLMs to improve the performance. TURL [12] first employed PLM for table understanding and Doduo [53] further improved the fine-tuning performance by introducing multi-task learning. Watchog [45] focused on improving the performance under low resource settings with self-supervised learning techniques [19]. TCN [58] and TabEmb [26] integrated graph neural network into fine-tuning to learn the inter-table information in addition to the table contents. There are other studies focused on devising new pre-training objectives for table datasets [22, 24, 31, 42] on the basis of BERT, which could also support table understanding tasks.

Recently LLMs have been widely adopted to support different table-related tasks [3, 21, 46, 69]. Some early table understanding studies [33, 35, 38] explored prompt strategies for table related tasks; ArcheType [20] focused on CTA and proposed some new benchmarking efforts in the context of LLMs. CENTS [61] reduced the prompt cost while ensuring good performance. This approach targeted saving LLM costs in one input prompt, while our proposed truncation approaches focus on reducing the number of table tokens in multiple training instances for LoRA based fine-tuning. Thus they have different design goals and trade-offs. ZTab [25] studied the prompt strategies under the zero-shot learning settings.

7.2 LLMs for Data Management Tasks

Many recent studies focused on improving the capability of LLMs to understand the general tabular structure. As is to be expected,

the most effective strategy is to perform a full fine-tuning over LLMs, such as TableGPT [39], Table-LLM-Specialist [63] and TableLLM [71]. However, given the enormous number of parameters, it is obvious that such methods are super expensive and require a huge amount of hardware resources. Compared with such efforts, our proposed work can reach a similar goal while significantly reducing the training overhead. Meanwhile, another line of studies focused on optimizing the prompt strategies to better use the reasoning power of LLMs over tabular data, such as Chain-of-Table [59], OpenTab [34] and TAP4LLM [54]. Such methods are orthogonal to our work.

7.3 Parameter-Efficient Fine-Tuning

Parameter-Efficient Fine-Tuning (PEFT) approaches aim to efficiently adapt large pretrained models to different applications by only fine-tuning a small number of additional parameters. Recently the most popular technique in this category is Low-rank Adaptation (LoRA) [27], which we use as a baseline and have modified for TSLoRA. There are many research efforts from the machine learning community in improving LoRA from different aspects, such as reducing the number of parameters [64] and adapting to longer contexts [7]. LoRA has also been widely applied to support different domains, such as image processing [1], natural language understanding [5], and graph analysis [66]. To the best of knowledge, the only previous study that adapted LoRA to table related tasks is TableLoRA [23]. TableLoRA only focused on the single task of table question answering (TQA) rather than obtaining a general purposed model for multiple table understanding tasks. Thus it cannot be easily extended to tasks other than TQA. Besides, there is also a significant difference in terms of both objective and methodology between our work and TableLoRA.

8 CONCLUSION AND FUTURE WORK

This paper proposed TSLoRA, an end-to-end framework for table understanding by fine-tuning LLMs with LoRA. TSLoRA enhances the capability of LLMs in various table understanding tasks with a lightweight fine-tuning process using a small number of training instances. To this end, we proposed an instruction tuning process to create training instances for different tasks from large table corpora within the same fine-tuning process. We further developed an effective two-stage training approach to incorporate the contextual and structural information of tables so as to optimize the fine-tuning process. Experimental results on several public benchmarking tasks demonstrated that TSLoRA can achieve promising effectiveness while reducing the overhead of fine-tuning. For future work, we plan to develop new techniques to further improve the performance for fine-tuning more table related reasoning tasks together.

ACKNOWLEDGMENTS

This work was partially funded by an NSERC Discovery Grant.

REFERENCES

[1] Sojung An, Junha Lee, Sujeong You, Nam Ik Cho, and Donghyun Kim. 2026. LoCA: Spatially-Aware Low-Rank Convolutional Adaptation of Vision Foundation Models. In *Computer Vision - ECCV 2026 - 19th European Conference, Malmö, Sweden, September 8-12, 2026, Proceedings, Part XXXV (Lecture Notes in Computer Science)*, Vol. 17035. Springer, 500–519.

[2] Gilbert Badaro, Mohammed Saeed, and Paolo Papotti. 2023. Transformers for Tabular Data Representation: A Survey of Models and Applications. *Trans. Assoc. Comput. Linguistics* 11 (2023), 227–249.

[3] Jan-Micha Bodensohn, Ulf Brackmann, Liane Vogel, Anupam Sanghi, and Carsten Binnig. 2025. Unveiling Challenges for LLMs in Enterprise Data Engineering. *Proc. VLDB Endow.* 19, 2 (2025), 196–209.

[4] Michael J. Cafarella, Alon Y. Halevy, Daisy Zhe Wang, Eugene Wu, and Yang Zhang. 2008. WebTables: exploring the power of tables on the web. *Proc. VLDB Endow.* 1, 1 (2008), 538–549.

[5] Jian Chen, Ruiyi Zhang, Yufan Zhou, Tong Yu, Franck Dernoncourt, Jiuxiang Gu, Ryan A. Rossi, Changyou Chen, and Tong Sun. 2025. SV-RAG: LoRA-Contextualizing Adaptation of MLLMs for Long Document Understanding. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

[6] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2020. TabFact: A Large-scale Dataset for Table-based Fact Verification. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net.

[7] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2024. LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[8] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao 'Kenneth' Huang, Bryan R. Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. Association for Computational Linguistics, 3697–3711.

[9] Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2022. HiTab: A Hierarchical Table Dataset for Question Answering and Natural Language Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*. Association for Computational Linguistics, 1094–1110.

[10] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *CoRR* abs/2412.19437 (2024).

[11] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR* abs/2501.12948 (2025).

[12] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2020. TURL: Table Understanding through Representation Learning. *Proc. VLDB Endow.* 14, 3 (2020), 307–319.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 4171–4186.

[14] Digital Research Alliance of Canada. 2026. Fir. https://docs.alliancecan.ca/wiki/Fir/en.

[15] Digital Research Alliance of Canada. 2026. Nibi. https://docs.alliancecan.ca/wiki/Nibi/en.

[16] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence* 5, 3 (2023), 220–235.

[17] Zhihao Ding, Yongkang Sun, and Jieming Shi. 2025. Retrieve-and-Verify: A Table Context Selection Framework for Accurate Column Annotations. *Proc. ACM Manag. Data* 3, 6 (2025), 358:1–358:27. https://doi.org/10.1145/3769823

[18] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, Seattle, WA, USA, June 18-23, 2023*. ACM, 69–75.

[19] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. *Proc. VLDB Endow.* 16, 7 (2023), 1726–1739.

[20] Benjamin Feuer, Yurong Liu, Chinmay Hegde, and Juliana Freire. 2024. ArcheType: A Novel Framework for Open-Source Column Type Annotation using Large Language Models. *Proc. VLDB Endow.* 17, 9 (2024), 2279–2292.

[21] Juliana Freire, Grace Fan, Benjamin Feuer, Christos Koutras, Yurong Liu, Eduardo Peña, Aécio S. R. Santos, Cláudio T. Silva, and Eden Wu. 2025. Large Language Models for Data Discovery and Integration: Challenges and Opportunities. *IEEE Data Eng. Bull.* 49, 1 (2025), 3–31.

[22] Michael R. Glass, Mustafa Canim, Alfio Gliozzo, Saneem A. Chemmengath, Vishwajeet Kumar, Rishav Chakravarti, Avi Sil, Feifei Pan, Samarth Bharadwaj, and Nicolas Rodolfo Fauceglia. 2021. Capturing Row and Column Semantics in Transformer Based Question Answering over Tables. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational*

Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021. Association for Computational Linguistics, 1212–1224.

[23] Xinyi He, Yihao Liu, Mengyu Zhou, Yeye He, Haoyu Dong, Shi Han, Zejian Yuan, and Dongmei Zhang. 2025. TableLoRA: Low-rank Adaptation on Table Structure Understanding for Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 22376–22391.

[24] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. Association for Computational Linguistics, 4320–4333.

[25] Ehsan Hoseinzade and Ke Wang. 2026. ZTab: Domain-Based Zero-Shot Annotation for Table Columns. In *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montreal, QC, Canada, May 4-8, 2026*. IEEE, 3625–3638.

[26] Ehsan Hoseinzade, Ke Wang, and Anandharaju Durai Raju. 2026. TabEmb: Joint Semantic-Structure Embedding for Table Annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2026, San Diego, California, United States, July 2-7, 2026*. Association for Computational Linguistics, 16620–16631.

[27] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

[28] Kevin Zeng Hu, Snehalkumar (Neil) S. Gaikwad, Madelon Hulsebos, Michiel A. Bakker, Emanuel Zgraggen, César A. Hidalgo, Tim Kraska, Guoliang Li, Arvind Satyanarayan, and Çagatay Demiralp. 2019. VizNet: Towards A Large-Scale Visualization Learning and Benchmarking Repository. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, CHI 2019, Glasgow, Scotland, UK, May 04-09, 2019*. 662.

[29] Madelon Hulsebos, Çagatay Demiralp, and Paul Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data* 1, 1 (2023), 30:1–30:17.

[30] Madelon Hulsebos, Kevin Zeng Hu, Michiel A. Bakker, Emanuel Zgraggen, Arvind Satyanarayan, Tim Kraska, Çagatay Demiralp, and César A. Hidalgo. 2019. Sherlock: A Deep Learning Approach to Semantic Data Type Detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*. ACM, 1500–1508.

[31] Hiroshi Iida, Dung Thai, Varun Manjunatha, and Mohit Iyyer. 2021. TABBIE: Pretrained Representations of Tabular Data. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*. Association for Computational Linguistics, 3446–3456.

[32] Ernesto Jiménez-Ruiz, Oktie Hassanzadeh, Kavitha Srinivas, Vasilis Efthymiou, and Jiaoyan Chen (Eds.). 2020. *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching co-located with the 18th International Semantic Web Conference, SemTab@ISWC 2019, Auckland, New Zealand, October 30, 2019*. CEUR Workshop Proceedings, Vol. 2553. CEUR-WS.org.

[33] Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. 2024. CHORUS: Foundation Models for Unified Data Discovery and Exploration. *Proc. VLDB Endow.* 17, 8 (2024), 2104–2114.

[34] Kezhi Kong, Jiani Zhang, Zhengyuan Shen, Balasubramaniam Srinivasan, Chuan Lei, Christos Faloutsos, Huzefa Rangwala, and George Karypis. 2024. OpenTab: Advancing Large Language Models as Open-domain Table Reasoners. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[35] Keti Korini and Christian Bizer. 2023. Column Type Annotation using ChatGPT. In *Joint Proceedings of Workshops at the 49th International Conference on Very Large Data Bases (VLDB 2023), Vancouver, Canada, August 28 - September 1, 2023 (CEUR Workshop Proceedings)*, Vol. 3462.

[36] Keti Korini, Ralph Peeters, and Christian Bizer. 2022. SOTAB: The WDC Schema.org Table Annotation Benchmark. In *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching, SemTab 2022, co-located with the 21st International Semantic Web Conference, ISWC 2022, Virtual conference, October 23-27, 2022 (CEUR Workshop Proceedings)*, Vol. 3320. CEUR-WS.org, 14–19.

[37] Oliver Lehmberg, Dominique Ritze, Robert Meusel, and Christian Bizer. 2016. A Large Public Corpus of Web Tables containing Time and Context Metadata. In *Proceedings of the 25th International Conference on World Wide Web, WWW 2016, Montreal, Canada, April 11-15, 2016, Companion Volume*. ACM, 75–76.

[38] Chenjie Li, Dan Zhang, and Jin Wang. 2024. LLM-assisted Labeling Function Generation for Semantic Type Detection. In *Proceedings of Workshops at the 50th International Conference on Very Large Data Bases, VLDB 2024, Guangzhou, China, August 26-30, 2024*. VLDB.org.

[39] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2024. Table-GPT: Table Fine-tuned GPT for Diverse Table Tasks. *Proc. ACM Manag. Data* 2, 3 (2024), 176.

[40] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, Jin Wang, Wataru Hirota, and Wang-Chiew Tan. 2021. Deep Entity Matching: Challenges and Opportunities. *ACM J. Data Inf. Qual.* 13, 1 (2021), 1:1–1:17.

[41] Girija Limaye, Sunita Sarawagi, and Soumen Chakrabarti. 2010. Annotating and Searching Web Tables Using Entities, Types and Relationships. *Proc. VLDB Endow.* 3, 1 (2010), 1338–1347.

[42] Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. 2022. TAPEX: Table Pre-training via Learning a Neural SQL Executor. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

[43] Meta AI. 2024. Llama 3.1 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md

[44] Meta AI. 2024. Llama 3.2 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md

[45] Zhengjie Miao and Jin Wang. 2023. Watchog: A Light-weight Contrastive Learning based Framework for Column Annotation. *Proc. ACM Manag. Data* 1, 4 (2023), 272:1–272:24.

[46] Avanika Narayan, Ines Chami, Laurel J. Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (2022), 738–746.

[47] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions - I. *Math. Program.* 14, 1 (1978), 265–294.

[48] Hyunjong Ok and Jaeho Lee. 2026. Lost in the Prompt Order: Revealing the Limitations of Causal Attention in Language Models. In *Findings of the Association for Computational Linguistics, ACL 2026, San Diego, California, United States, July 2-7, 2026*. Association for Computational Linguistics, 38566–38587.

[49] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023).

[50] OpenAI. 2026. OpenAI GPT-5 System Card. *CoRR* abs/2601.03267 (2026).

[51] Panupong Pasupat and Percy Liang. 2015. Compositional Semantic Parsing on Semi-Structured Tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July 26-31, 2015, Beijing, China, Volume 1: Long Papers*. The Association for Computer Linguistics, 1470–1480.

[52] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[53] Yoshihiko Suhara, Jinfeng Li, Yuliang Li, Dan Zhang, Çagatay Demiralp, Chen Chen, and Wang-Chiew Tan. 2022. Annotating Columns with Pre-trained Language Models. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*. ACM, 1493–1503.

[54] Yuan Sui, Jiaru Zou, Mengyu Zhou, Xinyi He, Lun Du, Shi Han, and Dongmei Zhang. 2024. TAP4LLM: Table Provider on Sampling, Augmenting, and Packing Semi-structured Data for Large Language Model Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024 (Findings of ACL)*, Vol. EMNLP 2024. Association for Computational Linguistics, 10306–10323.

[55] Yushi Sun, Hao Xin, and Lei Chen. 2023. RECA: Related Tables Enhanced Column Semantic Type Annotation Framework. *Proc. VLDB Endow.* 16, 6 (2023), 1319–1331. https://doi.org/10.14778/3583140.3583149

[56] Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *CoRR* abs/2507.06261 (2025).

[57] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. 5998–6008.

[58] Daheng Wang, Prashant Shiralkar, Colin Lockard, Binxuan Huang, Xin Luna Dong, and Meng Jiang. 2021. TCN: Table Convolutional Network for Web Table Interpretation. In *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*. ACM / IW3C2, 4020–4032.

[59] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2024. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[60] Xianjie Wu, Jian Yang, Linzheng Chai, Ge Zhang, Jiaheng Liu, Xeron Du, Di Liang, Daixin Shu, Xianfu Cheng, Tianzhen Sun, Tongliang Li, Zhoujun Li, and Guanglin Niu. 2025. TableBench: A Comprehensive and Complex Benchmark for Table Question Answering. In *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*. AAAI Press, 25497–25506.

[61] Guorui Xiao, Dong He, Jin Wang, and Magdalena Balazinska. 2025. CENTS: A Flexible and Cost-Effective Framework for LLM-Based Table Understanding. *Proc. VLDB Endow.* 18, 11 (2025), 4574–4587.

[62] Junjie Xing, Yeye He, Mengyu Zhou, Haoyu Dong, Shi Han, Lingjiao Chen, Dongmei Zhang, Surajit Chaudhuri, and H. V. Jagadish. 2025. MMTU: A Massive Multi-Task Table Understanding and Reasoning Benchmark. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

[63] Junjie Xing, Yeye He, Mengyu Zhou, Haoyu Dong, Shi Han, Dongmei Zhang, and Surajit Chaudhuri. 2025. Table-LLM-Specialist: Language Model Specialists for Tables using Iterative Fine-tuning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*. Association for Computational Linguistics, 35443–35460.

[64] Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng Zhang, and Qi Tian. 2024. QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[65] An Yang and et al. 2024. Qwen2 Technical Report. *CoRR* abs/2407.10671 (2024).

[66] Zhe-Rui Yang, Jindong Han, Chang-Dong Wang, and Hao Liu. 2025. GraphLoRA: Structure-Aware Contrastive Low-Rank Adaptation for Cross-Graph Transfer Learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1, KDD 2025, Toronto, ON, Canada, August 3-7, 2025*. ACM, 1785–1796.

[67] Andy B. Yoo, Morris A. Jette, and Mark Grondona. 2003. SLURM: Simple Linux Utility for Resource Management. In *Job Scheduling Strategies for Parallel Processing, 9th International Workshop, JSSPP 2003, Seattle, WA, USA, June 24, 2003, Revised Papers (Lecture Notes in Computer Science)*, Dror G. Feitelson, Larry Rudolph, and Uwe Schwiegelshohn (Eds.), Vol. 2862. Springer, 44–60.

[68] Dan Zhang, Yoshihiko Suhara, Jinfeng Li, Madelon Hulsebos, Çagatay Demiralp, and Wang-Chiew Tan. 2020. Sato: Contextual Semantic Type Detection in Tables. *Proc. VLDB Endow.* 13, 11 (2020), 1835–1848.

[69] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. 2024. Jellyfish: Instruction-Tuning Local Large Language Models for Data Preprocessing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*. Association for Computational Linguistics, 8754–8782.

[70] Shuo Zhang and Krisztian Balog. 2017. EntiTables: Smart Assistance for Entity-Focused Tables. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval, Shinjuku, Tokyo, Japan, August 7-11, 2017*. ACM, 255–264.

[71] Xiaokang Zhang, Sijia Luo, Bohan Zhang, Zeyao Ma, Jing Zhang, Yang Li, Guanlin Li, Zijun Yao, Kangli Xu, Jinchang Zhou, Daniel Zhang-Li, Jifan Yu, Shu Zhao, Juanzi Li, and Jie Tang. 2025. TableLLM: Enabling Tabular Data Manipulation by LLMs in Real Office Usage Scenarios. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025 (Findings of ACL)*, Vol. ACL 2025. Association for Computational Linguistics, 10315–10344.

[72] Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh M. Patel. 2024. ReAcTable: Enhancing ReAct for Table Question Answering. *Proc. VLDB Endow.* 17, 8 (2024), 1981–1994.